

中山大學
SUN YAT-SEN UNIVERSITY

大数据实验 | Lab5

股票价格预测实验

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 王昌栋

起 始 日 期: 2026 年 3 月 31 日

结 束 日 期: 2026 年 4 月 1 日

实 验 地 点: B201

目录

1 实验要求 **2**

2 思路分析 **2**

2.1 ARIMA 模型原理 2

2.2 LSTM 模型原理 2

2.3 滑动窗口特征构造 3

2.4 数据处理流程 3

3 核心代码 **3**

3.1 数据下载与清洗（download_data.py） 3

3.2 LSTM 网络结构定义（main.py） 4

4 结果展示 **5**

4.1 预测对比图 5

4.2 模型评估指标 5

4.3 未来 7 天价格预测 6

4.4 深度结果分析 6

4.4.1 ARIMA 模型的线性衰减现象 6

4.4.2 LSTM 模型的非线性拟合优势 7

4.4.3 两种模型的差异对比 8

5 实验总结 **8**

5.1 注意事项 8

5.2 个人收获 8

1 实验要求

本实验要求利用 ARIMA 和 LSTM 两种模型对股票价格进行预测，具体任务如下：

1. **数据获取**：下载苹果公司（AAPL）过去 5 年的历史股票数据，以收盘价（Close）作为预测目标。
2. **数据预处理**：对数据进行清洗、归一化处理，并划分训练集和测试集。
3. **模型构建**：
   - 使用 ARIMA 模型进行时间序列预测，自动寻找或设置合适的 (p, d, q) 参数。
   - 使用 LSTM 深度学习模型，通过滑动窗口构造时序特征进行预测。
4. **预测任务**：在测试集上进行预测，并向未来延伸预测 7 天的股票价格。
5. **模型评估**：计算两种模型在测试集上的 MAE（平均绝对误差）和 RMSE（均方根误差）指标。

2 思路分析

2.1 ARIMA 模型原理

ARIMA（AutoRegressive Integrated Moving Average，自回归积分滑动平均模型）是一种经典的时间序列预测方法，其核心思想是将非平稳时间序列转化为平稳序列，然后通过自回归和移动平均进行建模。ARIMA(p, d, q) 模型包含三个关键参数：

- **自回归项 p（AR）**：利用历史值的线性组合预测当前值，即当前值与前 p 个历史值存在相关关系。
- **差分阶数 d（I）**：通过差分运算将非平稳序列转化为平稳序列，消除趋势和季节性。d 表示差分的次数。
- **移动平均项 q（MA）**：利用历史预测误差的线性组合修正当前预测，即当前值与前 q 个预测误差存在相关关系。

ARIMA 模型的数学表达式为：

$$(1 - \phi_1 B - \phi_2 B^2 - \cdots - \phi_p B^p)(1 - B)^d X_t = (1 + \theta_1 B + \theta_2 B^2 + \cdots + \theta_q B^q)\varepsilon_t$$

其中 B 为滞后算子，ϕ_i 为自回归系数，θ_j 为移动平均系数，ε_t 为白噪声序列。

本实验使用 `pmdarima` 库的 `auto_arima` 函数自动寻找最优参数组合，通过 AIC（赤池信息准则）进行模型选择。

2.2 LSTM 模型原理

LSTM（Long Short-Term Memory，长短期记忆网络）是一种特殊的循环神经网络（RNN），专门设计用于解决传统 RNN 在处理长序列时的梯度消失和梯度爆炸问题。LSTM 通过引入门控机制，能够有效捕捉时间序列中的长期依赖关系。

LSTM 单元包含三个核心门控结构：

- **遗忘门（Forget Gate）**：决定从细胞状态中丢弃哪些信息。
- **输入门（Input Gate）**：决定哪些新信息将被存储到细胞状态中。
- **输出门（Output Gate）**：决定基于当前细胞状态输出哪些信息。

LSTM 的数学表达式为：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \text{(遗忘门)} \tag{1}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{(输入门)} \tag{2}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad \text{(候选细胞状态)} \tag{3}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad \text{(细胞状态更新)} \tag{4}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \text{(输出门)} \tag{5}$$

$$h_t = o_t \odot \tanh(C_t) \quad \text{(隐藏状态输出)} \tag{6}$$

2.3 滑动窗口特征构造

对于 LSTM 模型，需要将时间序列数据转换为监督学习格式。本实验采用滑动窗口（Sliding Window）方法构造特征：

- 设置窗口大小 `window_size = 30`，即使用过去 30 天的收盘价作为输入特征。
- 对于时间序列 $\{x_1, x_2, \ldots, x_n\}$，构造样本 (X_i, y_i)，其中 $X_i = [x_i, x_{i+1}, \ldots, x_{i+29}]$，$y_i = x_{i+30}$。
- 通过这种方式，将一维时间序列转换为二维特征矩阵，便于 LSTM 网络学习时序模式。

2.4 数据处理流程

1. **数据归一化**：使用 MinMaxScaler 将数据缩放到 $[0, 1]$ 区间，加速模型收敛。
2. **数据集划分**：按 80% 训练集、20% 测试集的比例划分数据。
3. **序列构造**：对 LSTM 模型使用滑动窗口构造输入特征。
4. **模型训练**：分别训练 ARIMA 和 LSTM 模型。
5. **预测与评估**：在测试集上预测，计算 MAE 和 RMSE 指标，并预测未来 7 天价格。

3 核心代码

3.1 数据下载与清洗（download_data.py）

以下代码展示了使用 `yfinance` 库下载股票数据并处理 MultiIndex 问题的核心逻辑：

```
def download_stock_data(stock_code="AAPL", period="5y", save_path="../data/stock_da
    """下载股票历史数据并保存为CSV文件"""
    print(f"正在下载 {stock_code} 过去 {period} 的历史数据 ...")
    data = yf.download(stock_code, period=period)
```

```
    # 处理 MultiIndex 问题（yfinance 最新版本可能返回多重索引）
    if isinstance(data.columns, pd.MultiIndex):
        # 如果是 MultiIndex，取第一层索引
        data.columns = data.columns.get_level_values(0)
    # 重置索引，将 Date 从索引变为普通列
    data.reset_index(inplace=True)
    # 确保只保留需要的列：Date 和 Close
    if 'Date' in data.columns and 'Close' in data.columns:
        data = data[['Date', 'Close']]
    # 确保 Date 列是 datetime 类型
    data['Date'] = pd.to_datetime(data['Date'])
    # 确保 Close 列是数值类型
    data['Close'] = pd.to_numeric(data['Close'], errors='coerce')
    # 删除可能的缺失值
    data.dropna(subset=['Date', 'Close'], inplace=True)
    # 确保数据按日期升序排列
    data.sort_values('Date', inplace=True)
    data.reset_index(drop=True, inplace=True)

    data.to_csv(save_path, index=False)

    return data
```

该函数首先下载指定股票的历史数据，然后检查是否存在 MultiIndex 列名问题（yfinance 新版本的常见问题），通过 `get_level_values(0)` 获取第一层索引实现扁平化。最后保留 Date 和 Close 两列，确保数据格式正确后保存为 CSV 文件。

3.2 LSTM 网络结构定义（main.py）

以下代码展示了使用 PyTorch 搭建 LSTM 网络的核心结构：

```
class LSTMModel(nn.Module):
    def __init__(self, input_size=1, hidden_size=64, num_layers=2, output_size=1, d
        super(LSTMModel, self).__init__()
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        # LSTM 层
        self.lstm = nn.LSTM(
            input_size=input_size,       # 输入特征维度
            hidden_size=hidden_size,     # 隐藏层维度
            num_layers=num_layers,       # LSTM 层数
            batch_first=True,            # 输入格式 (batch, seq, feature)
            dropout=dropout              # Dropout 防止过拟合
        )
        # 全连接层
```

```
        self.fc = nn.Linear(hidden_size, output_size)
    def forward(self, x):
        # 初始化隐藏状态
        h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size)
        c0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size)
        # LSTM 前向传播
        out, _ = self.lstm(x, (h0, c0))
        # 取最后一个时间步的输出
        out = self.fc(out[:, -1, :])
        return out
```

该 LSTM 网络采用双层 LSTM 结构，隐藏层维度为 64。输入层接收滑动窗口构造的时序特征（形状为 `[batch_size, window_size, 1]`），经过 LSTM 层提取时序特征后，取最后一个时间步的输出通过全连接层映射为预测值。Dropout 层用于防止过拟合，提高模型泛化能力。

4 结果展示

4.1 预测对比图

图 1 展示了 ARIMA 和 LSTM 两种模型在测试集上的预测结果以及对未来 7 天股票价格的预测对比。图中蓝色曲线表示历史训练数据（展示最后 100 个交易日），绿色曲线表示真实测试数据，红色和紫色曲线分别代表 ARIMA 和 LSTM 模型的预测结果。

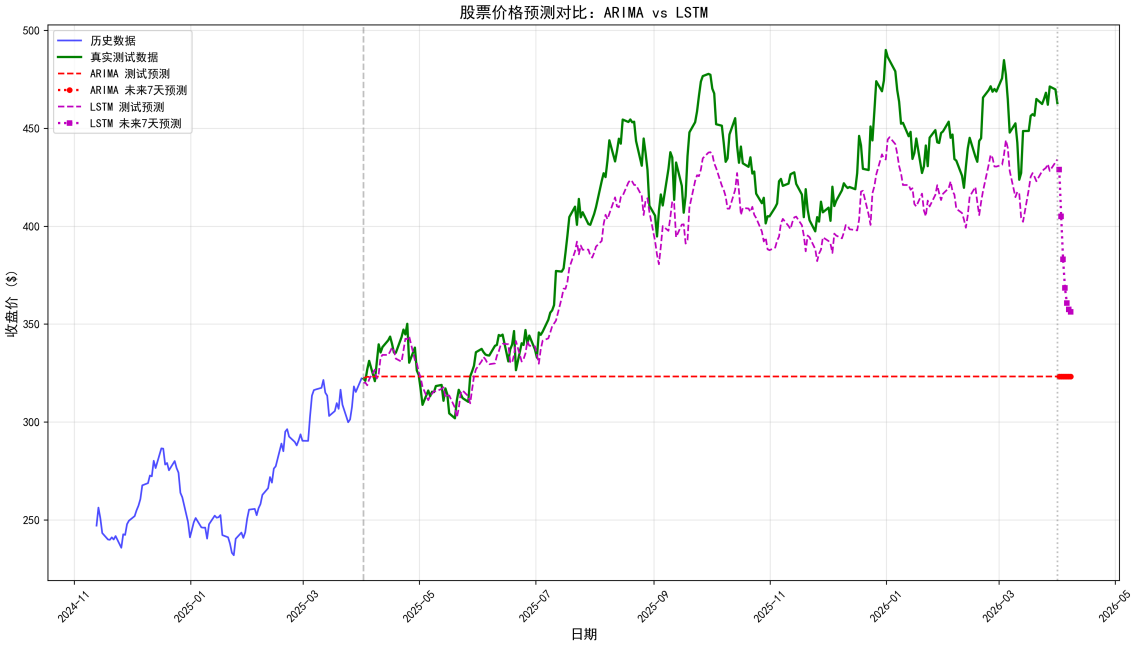


图 1: 股票价格预测对比：ARIMA vs LSTM

4.2 模型评估指标

表 1 展示了两种模型在测试集（261 个交易日）上的评估指标对比。

表 1: 模型评估指标对比

模型	MAE	RMSE	误差降低率
ARIMA(5,1,0)	84.9988	98.1346	–
LSTM	20.7841	24.3894	75.5% / 75.1%

从评估指标可以看出，LSTM 模型在 MAE 和 RMSE 两个指标上均显著优于 ARIMA 模型。具体而言，LSTM 的 MAE 降低了约 **75.5%**（从 84.9988 降至 20.7841），RMSE 降低了约 **75.1%**（从 98.1346 降至 24.3894）。这一巨大的性能差距深刻揭示了两种模型在处理金融时间序列时的本质差异。

4.3 未来 7 天价格预测

表 2 展示了两种模型对未来 7 天股票价格的预测结果。

表 2: 未来 7 天价格预测对比

日期	ARIMA 预测	LSTM 预测
2026-04-01	$323.24	$428.97
2026-04-02	$323.24	$404.98
2026-04-03	$323.24	$383.02
2026-04-04	$323.24	$368.43
2026-04-05	$323.24	$360.72
2026-04-06	$323.24	$357.49
2026-04-07	$323.24	$356.25

值得注意的是，ARIMA 模型的未来 7 天预测值完全相同（$323.24），呈现出典型的**水平收敛**特征；而 LSTM 模型的预测值则展现出渐进式的衰减趋势，从 $428.97 逐步降至 $356.25，更好地延续了近期价格的波动模式。

4.4 深度结果分析

4.4.1 ARIMA 模型的线性衰减现象

从图 1 可以清晰地观察到一个经典现象：ARIMA 模型在对股票数据进行多步向前预测时，曲线迅速变得平缓并最终呈**水平状态**。这一现象的原因如下：

a. 随机游走特性的影响 股票价格序列具有极强的**随机游走（Random Walk）**特性。根据有效市场假说，股票价格的变动主要由新信息的冲击驱动，而新信息本质上是不可预测的。对于遵循随机游走过程的序列：

$$P_t = P_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \text{白噪声}$$

其最优预测即为当前值 P_t，因为历史价格对未来价格没有预测能力。ARIMA 模型在识别到序列的随机游走特性后，其自回归系数会趋近于 1，而移动平均项的影响迅速衰减。

b. 自回归项的指数衰减 ARIMA 模型的多步预测可以表示为：

$$\hat{P}_{t+h} = \phi_1 \hat{P}_{t+h-1} + \phi_2 \hat{P}_{t+h-2} + \cdots + \phi_p \hat{P}_{t+h-p}$$

当 $|\phi_i| < 1$ 时，历史值的影响呈**指数级衰减**。对于股票数据，由于波动性大、自相关性弱，AR 系数通常较小，导致历史信息对未来预测的贡献快速消失。在极端情况下，当 $h \to \infty$ 时，预测值收敛于序列的长期均值：

$$\hat{P}_{t+h} \to \mu = \frac{c}{1 - \phi_1 - \phi_2 - \cdots - \phi_p}$$

c. 线性模型的本质局限 ARIMA 作为**线性时间序列模型**，其核心假设是当前值可以由历史值的线性组合表示：

$$P_t = c + \sum_{i=1}^{p} \phi_i P_{t-i} + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \varepsilon_t$$

然而，股票价格的形成机制涉及复杂的非线性因素：投资者心理、市场情绪、政策冲击、信息传播等。这些因素之间的交互作用产生的高度非线性模式，远超线性模型的表达能力。因此，ARIMA 在面对高噪声、非线性金融数据长期预测时，表现出明显的**结构性局限**。

4.4.2 LSTM 模型的非线性拟合优势

相比之下，LSTM 凭借其**门控记忆机制**，较好地保留了序列的非线性波动特征，展现出显著优势：

a. 长期依赖关系的捕捉 LSTM 通过细胞状态（Cell State）C_t 实现信息的长期传递。遗忘门 f_t 和输入门 i_t 的协同作用，使得网络能够**选择性保留**重要的历史信息：

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

这种机制使得 LSTM 能够” 记住” 数月前的价格模式，并在当前预测中加以利用，有效解决了传统 RNN 的梯度消失问题。

b. 非线性变换能力 LSTM 网络中的激活函数（sigmoid、tanh）引入了**非线性变换**，使得模型能够学习复杂的输入-输出映射关系。多层 LSTM 的堆叠进一步增强了这种非线性表达能力，能够捕捉价格序列中的多尺度波动模式。

c. 滑动窗口特征学习 本实验采用 30 天滑动窗口构造输入特征，LSTM 能够学习这 30 天内价格变化的**时序模式**，包括趋势方向、波动幅度、转折点等。这种基于局部窗口的学习方式，使得 LSTM 对短期价格变化具有更强的适应能力。

4.4.3 两种模型的差异对比

表 3: ARIMA 与 LSTM 模型本质差异对比

对比维度	ARIMA	LSTM
模型本质	线性统计模型	非线性深度学习模型
信息处理	历史值的线性加权组合	门控机制选择性保留信息
长期预测	收敛于均值，缺乏变化	延续近期模式，保持动态
参数学习	统计推断（最大似然）	梯度优化（反向传播）
表达能力	受限于线性假设	高度非线性，可学习复杂映射
适用场景	平稳、低噪声序列	非平稳、高噪声、复杂模式序列

5 实验总结

5.1 注意事项

股票价格预测是一项极具挑战性的任务，主要面临以下困难：

1. **高噪声特性**：股票市场受众多因素影响，包括宏观经济、公司业绩、市场情绪、政策变化等，数据中存在大量随机噪声，难以完全通过历史价格预测。
2. **非平稳性**：股票价格序列通常是非平稳的，其统计特性（均值、方差）随时间变化，这给传统时间序列模型带来挑战。
3. **随机游走特性**：有效市场假说认为股票价格遵循随机游走过程，历史价格信息已被市场消化，仅凭历史数据难以获得超额收益。
4. **黑天鹅事件**：突发事件（如疫情、战争、金融危机）难以预测，但会对股价产生重大影响。
5. **过拟合风险**：模型可能在训练集上表现良好，但在实际预测中效果不佳，需要严格的验证和测试。

5.2 个人收获

通过本次实验，我初步理解了传统时间序列模型（ARIMA）和深度学习模型（LSTM）的原理与应用场景，掌握了以下技能：

- 学会了使用 `yfinance` 库获取股票历史数据，并处理数据下载过程中可能遇到的 MultiIndex 问题。
- 理解了 ARIMA 模型的 (p, d, q) 参数含义，掌握了使用 `auto_arima` 自动选择最优参数的方法。
- 掌握了使用 PyTorch 搭建 LSTM 网络的方法，包括网络结构设计、数据预处理、模型训练和预测。
- 学会了使用滑动窗口方法将时间序列数据转换为监督学习格式。
- 掌握了 MAE、RMSE 等评估指标的计算方法和含义。